FS Organizer

User manual

Version 0.51.0

Contents

1 What this program does

FS Organizer turns Microsoft Flight Simulator addons on and off without moving a single file.

Your addons live in a folder of your own, outside the simulator, which the program calls a **library**. Its subfolders become **categories**. Enabling an addon creates a link inside a folder the simulator reads, which the program calls a **destination**; disabling removes that link and nothing else. The real files never leave the library, and that is why swapping tens of gigabytes of content is instant.

The link is an NTFS directory junction. To the simulator it is the folder; to you it is an entry FS Organizer creates and removes.

1.1 What the program does not store

Whether an addon is enabled **is never written down anywhere**. On every scan the program reads the destinations from the disk and answers for what it found there. That has a practical consequence: moving folders around from the outside, with Explorer or with another program's installer, never leaves the app saying one thing while the simulator does another. Read again and it is current.

1.2 Profile

A **profile** is one simulator installation with its destinations and its libraries. Anyone running Flight Simulator 2020 and 2024 on the same machine has two profiles, and the program only touches what is in use: marking another one active is what switches the target of everything.

1.3 Link type

In **Options**, under **Links**, the program offers two types.

Directory junction
Needs no administrator and crosses local volumes. It is the default, and the only path that has been tested.

Symbolic link
Only for a library on a network path, where the junction does not reach. It needs a Windows privilege; without it the program explains the refusal instead of failing quietly.

Switching the type changes new links only. The ones that already exist stay as they are.

2 Presets

This is the first subject of the manual because it is the one nobody gets right by trial. A preset looks like a list of addons and is not: it is a list of addons **plus an apply mode**, and the same preset produces three different results depending on the mode.

2.1 What a preset holds

A preset keeps which addons stay enabled. You enable what you want to fly and keep that combination under a name, with **New from the enabled ones**.

The **Content** column says how many addons it names and how many categories they sit in. The **Changed** column says when it was last written.

A preset can also govern the startup entries of the simulator, and section 2.6 covers that.

2.2 A satisfied preset

The **Would change** column answers how many addons would change state if you applied that preset now. When that number is zero, the table says **Satisfied**.

Satisfied is always measured as Replace, even when you mean to apply it some other way. A preset naming three addons, all of them on, shows as satisfied only if **nothing beyond them** is on. If you have a fourth addon enabled that the preset does not name, that preset is not satisfied, because applying it as Replace would turn the fourth one off.

This is deliberate: the question the column answers is “is my installation the way this preset describes it?”, and the only reading in which a preset describes the whole installation is Replace.

2.3 The three apply modes

The **Apply as** selector chooses between three readings of the same preset.

Replace
Leaves only what the preset enables. It applies what the preset names and turns off every enabled addon it does not name.

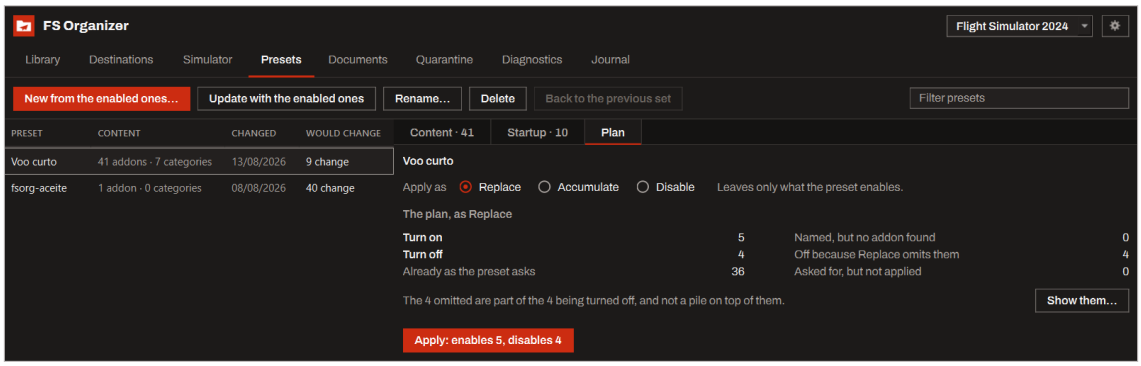
Accumulate
Enables what the preset names, without touching the rest. It applies what the preset names and leaves everything else alone.

Disable
Disables what the preset enables. It is the list read backwards: the addons the preset would turn on end up off, and the entries it already asked to turn off are ignored.

Before you apply, the **Plan** panel shows what is about to happen, in five groups that add up to the whole count:

- **Turn on**: addons the preset names that are off right now.

- **Turn off**: addons that are going out, whether because the preset asks or because Replace omits them.
- **Already as the preset asks**: nothing happens to them.
- **Named, but no addon found**: the preset names an addon that is no longer in the library. The program invents nothing and does not fail over it; it lists which ones.
- **Asked for, but not applied**: startup entries the preset asks for that could not be switched.



The plan of the “Voo curto” preset, read as Replace. The five groups add up to the whole count, and the sentence under them says the 4 omitted are inside the 4 about to be turned off.

2.4 What falls off by omission in Replace

Applying as Replace turns off the addons that are enabled right now and that the preset does not name. They show up under **Off because Replace omits them**, and the **Show them** button opens the list.

The app says, beside that list, that **they are part of what the plan already counts as turned off, and not a pile on top of it**. If the plan says “disables 12” and the omitted list has 5 names, the total is still 12, with 5 of them entering by omission and 7 because the preset asked. Adding the two numbers gives 17 and is wrong.

2.5 The return preset

Before applying any preset, the program **writes down what is enabled at that moment** and saves it as the profile’s return preset. The **Back to the previous set** button applies that note.

If the note cannot be written, because the presets folder is full or protected, **nothing is applied**. The program would rather not make the change than make it with no way back.

The return is not the same as **Undo the last batch**, and the difference matters:

Undo the last batch
Undoes exactly the batch you just applied. An import or a quarantine operation done afterwards consumes the undo, and it stops being available.

Back to the previous set
Applies the return preset, which is a preset like any other. It is still there after an import or a quarantine, because it does not depend on the journal.

Applying a preset is a single batch: **Undo the last batch** takes it all back at once, not addon by addon.

2.6 Presets and startup entries

On the preset's **Startup** tab, a checkbox makes that preset also manage the programs the simulator opens with itself. Checking it captures the entries enabled at that moment, and from then on you turn each one on or off there.

If startup management is off in **Options**, the entries the preset asks for **are not applied**, and the program says how many. The rest of the preset is applied normally.

If an entry the preset names is no longer in the simulator's file, it is listed instead of being recreated. The program never adds a startup entry.

3 First setup

On the first run the program looks for the simulator installations on the machine and asks for two things.

The simulator
Choose the installation this profile will manage, from the list it found, or point at a folder by hand. If the folder you point at does not look like a simulator destination, which is usually called `Community`, the program says so and uses it anyway.

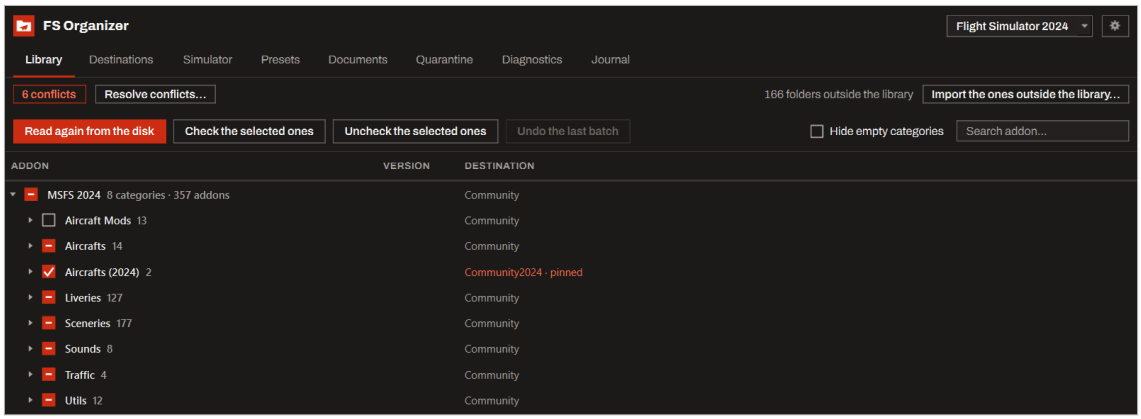
The library
Choose the root folder where your addons are kept, outside the simulator. Its subfolders become categories.

When you register a very deep library, the program shows how many characters that root consumes and how many are left for the addon, next to a short root for comparison. Nothing is blocked: addons that go past the Windows limit say so when it matters, and **Diagnostics** keeps the longest path of each library.

Anyone coming from MSFS Addons Linker can bring its configuration over in **Options**, through **Import from MSFS Addons Linker**. Nothing is moved or deleted: importing registers the library, the categories and the presets that are not here yet.

4 Library

The **Library** tab is the tree of your addons, grouped by the categories that are the subfolders of the root. The checkbox on each row turns it on and off.



The library, with the categories open and the panel of the selected addon on the right.

4.1 The columns

Addon
The folder name.

Version
What the addon's `manifest.json` declares.

Enabled
Whether a link to it exists in a destination right now.

Destination
Which destination it is linked in, and whether that destination is pinned.

Target exists
Whether the link finds the folder. A "no" here is a broken link.

Three marks appear on a row when something is out of place: **No target**, **In conflict** and **Two copies**. Each one has a section in this manual.

4.2 Enabling and disabling in bulk

Selecting several rows and using **Check the selected ones** or **Uncheck the selected ones** does it all in one batch. Past a certain number, the program asks first.

If an addon's spot is taken by another addon of yours, the program does not refuse: it offers the **swap**, which goes out and comes in as one operation, with one line in the journal and one undo.

4.3 Categories

The tree's context menu creates, renames and deletes categories, and moves addons between them. Only an empty category can be deleted.

Suggest categories reads rules about the folder name and about the `manifest.json` and proposes moving whatever is out of place. The rules that get it right on their own come checked; the livery rule comes unchecked because it is often wrong, so check it before applying.

A folder counts as a category while it groups addons, and the program writes a hidden `.fsorg-category` inside the ones it knows about so that a category which loses its last addon keeps being one. It marks them when it registers the library, and **Categories** in the options marks them again later.

A folder that groups no addon and carries no `manifest.json` of its own is treated as an addon, so it can be enabled, moved and deleted like any other. That is what a converted package the simulator writes beside yours looks like, and what a scenery object library that ships without a manifest looks like too.

4.4 Pinning the destination

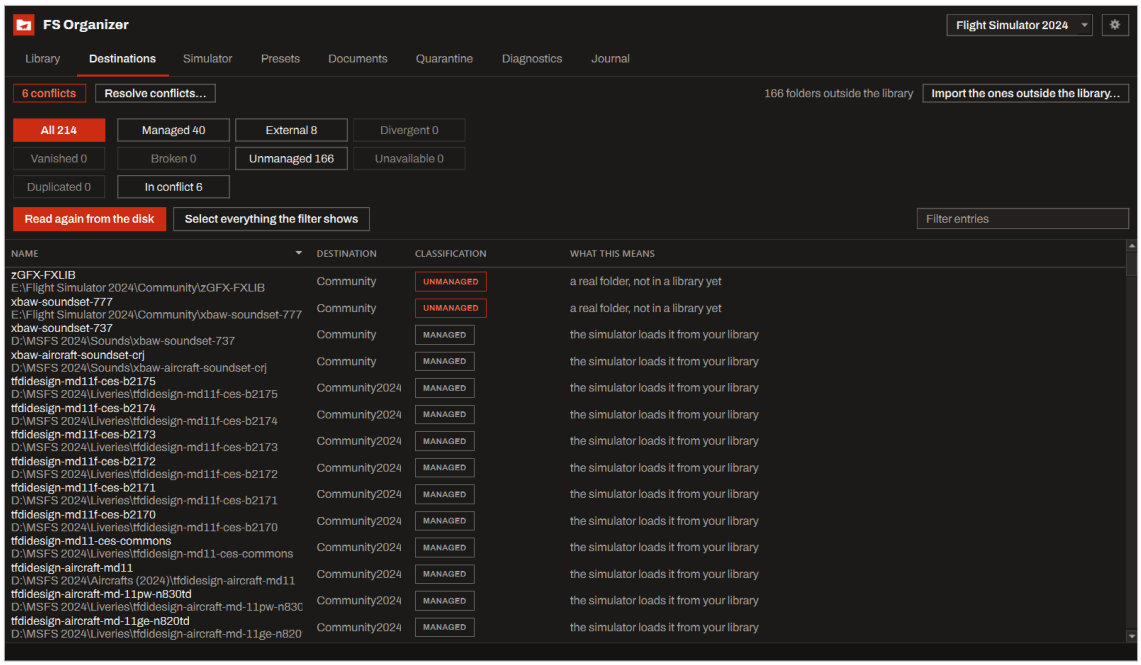
By default an addon is linked in the profile's default destination. **Pin the destination** ties an addon or a whole category to a specific one. When a pinning names a folder that has stopped being a destination of the profile, the program says so and uses the default destination while that is true, without deleting the pinning from the configuration.

4.5 Repointing

When a link exists but points at the wrong place, or at a folder that is gone, **Repoint to the library** fixes it. Repairing never touches the real files: only the reparse node is rewritten.

5 Destinations

The **Destinations** tab shows everything inside the folders the simulator reads, including what is not yours. Each entry gets a classification.



The destinations, with the classification filter at the top and the count of each kind in the footer.

Managed
The simulator loads it from your library. This is the normal state.

External
A real folder, not in a library yet. Another program installed it there.

Divergent
The other program took its folder back, so there are two copies.

Vanished
The library copy is gone, taken by the other program. There is nothing to repair: the content no longer exists on this machine.

Broken
The target of the link does not exist.

Unavailable
The volume is not there right now. It can come back, which is why the program offers no cleanup.

Unmanaged
A real folder nobody has taken over.

Duplicated
Linked in more than one destination.

The filter at the top narrows the list by classification, and **Select everything the filter shows** marks exactly what is visible.

5.1 Taking over another program's folder

Import this folder moves the folder into your library and leaves a link in its place. From then on it is an addon of yours like any other.

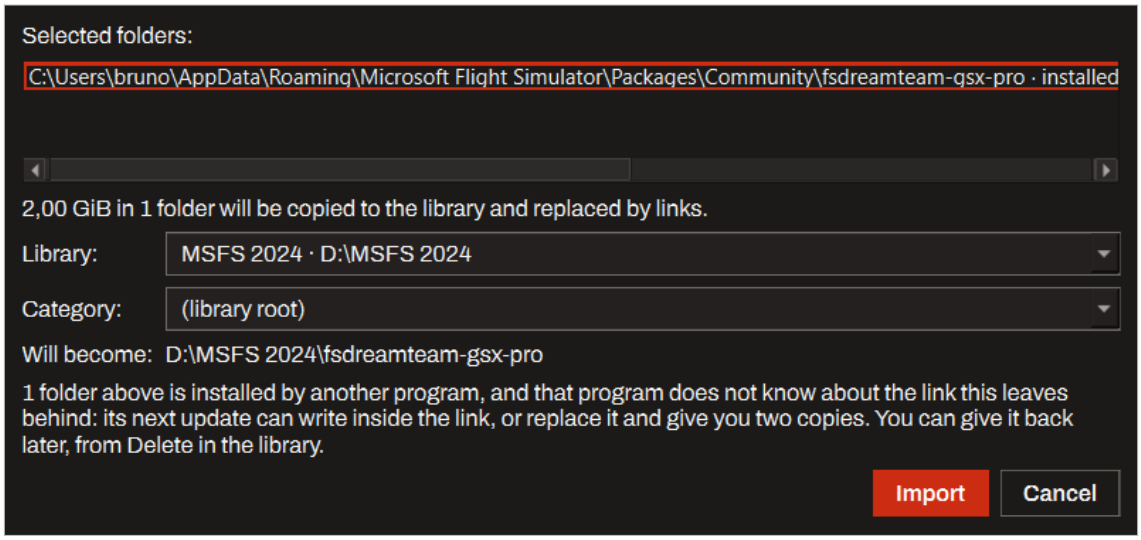
The program is explicit about the risk: **the other program does not know about the link**. Its next update can write inside the link, or replace it with a real folder and give you two copies. Nothing in FS Organizer can stop that. When it happens, the entry shows up as **Divergent** and the program offers you the choice of which copy stays.

You can give the addon back to the program that installed it later, through **Delete from the library**.

6 Importing

Importing copies a real folder into the library and leaves a link in its place. The original folder is only removed after the check.

The operation has five steps, and the journal records each one:



The import dialog, which says how much is about to be copied, into which library and category, and where the addon will end up.

1. Copy to the staging area.
2. Check the copy.
3. Put the copy in place.
4. Remove the source folder.
5. Create the link in the destination.

While the copy runs, a window shows which folder it is on, how far it has got and a **Cancel** button. Cancelling deletes nothing: what was already copied stays where it is, for the resume.

6.1 Check after copying

In **Options**, under **Links**, the check comes in two forms.

By structure
Checks the count and the size of every file. It is the default.

By hash
Reads both sides in full and compares the bytes, which catches a change the size hides. It makes an import several times slower, and the number of files weighs more than their size.

If the copy does not match, nothing is removed.

6.2 Half finished imports

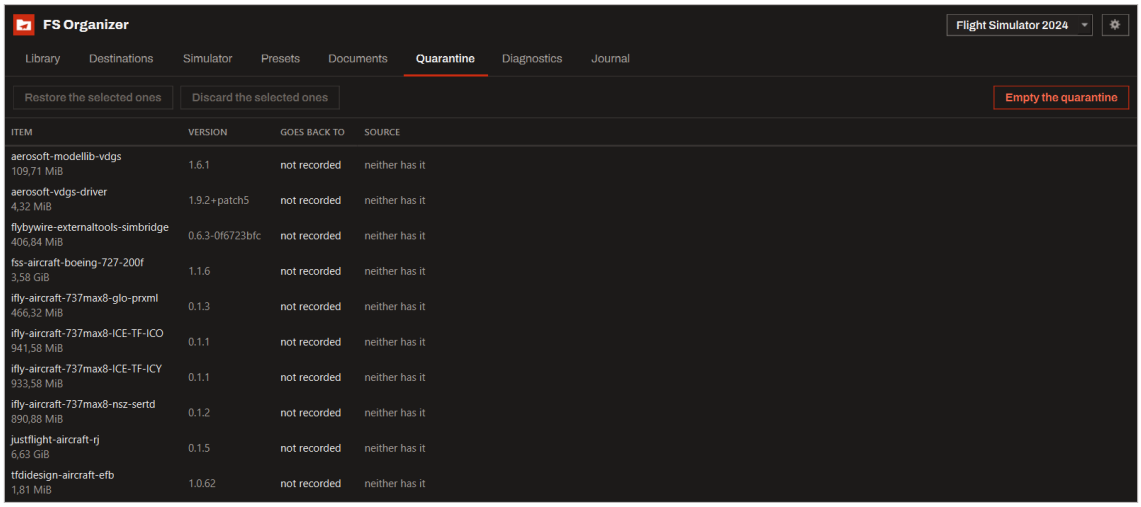
If an import is interrupted, the next run offers three ways out: resume, discard the half finished copy, or leave it as it is. The original files are still where they were, and nothing was removed from the destination.

When the half that stopped is a conflict resolution, only discarding is offered, because both copies are still where they were.

7 Quarantine

When two copies of the same addon fight over the same name, the losing one comes to the quarantine instead of being deleted. **Nothing leaves the quarantine without you saying so**, and nothing is deleted on the way in.

Beside each item the program records where it came from. That record is independent of the journal, and it is what allows an item to be restored even after the journal has been deleted. The table shows both sources, **The record says** and **The Journal says**, and flags when they disagree.



The quarantine, with the origin of each item and the flag for when the record and the journal disagree.

Restore sends each folder back to where it came from. Nothing is overwritten: what would collide is listed with both versions, and replacing puts the occupant in the quarantine with its own origin recorded. **Discard** erases from the disk for good, and asks first.

8 Deleting an addon from the library

Delete offers three routes, and says up front when one of them will not work.

Move to the Recycle Bin
You can put it back through Windows itself. The program refuses this route when the selection does not fit in the Recycle Bin of that volume, or when the addon holds a path past the 260 characters the Recycle Bin stops at. Windows may quietly evict older items from the Bin to make room, and the program cannot prevent that.

Delete permanently
It does not come back. Not through the Recycle Bin either.

Give it back to the other program
Only offered for an addon that came from another program. The bytes move back to the folder that program manages, and the library copy goes away with the import that made it.

Turning the addon off is part of the deletion and happens because you asked for it: the link goes with the addon.

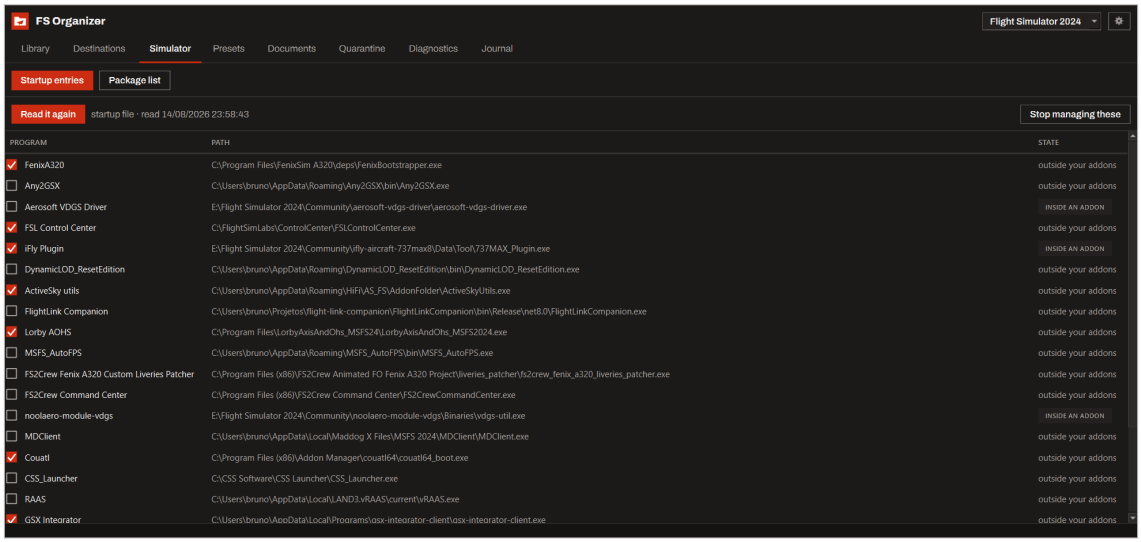
9 Simulator

The **Simulator** tab has two panels, and both write to files that belong to the simulator. In both, the program keeps one backup copy of the file beside it, and neither writes while the simulator is running.

9.1 Startup entries

The simulator has a file that decides what opens with it. The **Startup entries** panel lists those programs and turns each one on or off, without you editing XML.

The program **writes one thing there: the switch of an entry that already exists**. It never adds, removes or reorders.



The startup entries, with the program each one launches and its switch.

This management starts off. While it is off, the program neither reads nor writes that file.

Disabling an addon that carries a live startup entry makes the program warn first: leaving the entry on would have the simulator keep trying to launch a program that will not be there. Turning both off is a single operation, one line in the journal, undone as one.

9.2 Package list

The simulator ships airports of its own. When an airport of yours covers the same place as one of them, the **Package list** panel shows the pair and lets you turn the simulator's one off.

Turning it off writes **one value** in the package list. The program adds nothing, removes nothing and reorders nothing, and it refuses an entry that does not carry the attribute instead of adding it, because invalid XML makes the simulator wipe the whole list.

Between two addons **of your own**, the program only shows the pair and turns nothing off: turning one off is enabling and disabling, which you already do on the **Library** tab.

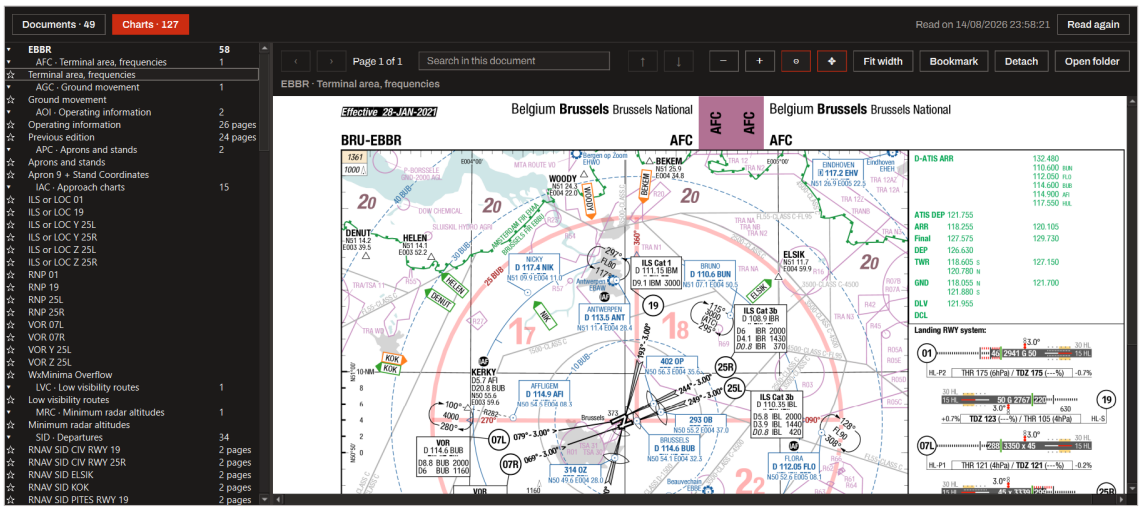
This panel also starts off, and while it is off the file is neither read nor written.

10 Documents

The **Documents** tab finds the PDFs your addons carry and opens them inside the program. Reading the library walks every addon looking for PDFs, and what it finds is written down so the next time is instant.

What it finds is split in two: **Charts** and **Manuals**. One addon can carry one manual and a hundred and fifty charts, which is why the count comes before the list. Charts are grouped by airport and by type when the addon ships a catalogue describing them.

The reader has an outline, search inside the document, fit to width and bookmarks. A bookmark you do not name takes the page number. The **Detach** button pulls the reader out into a separate window.

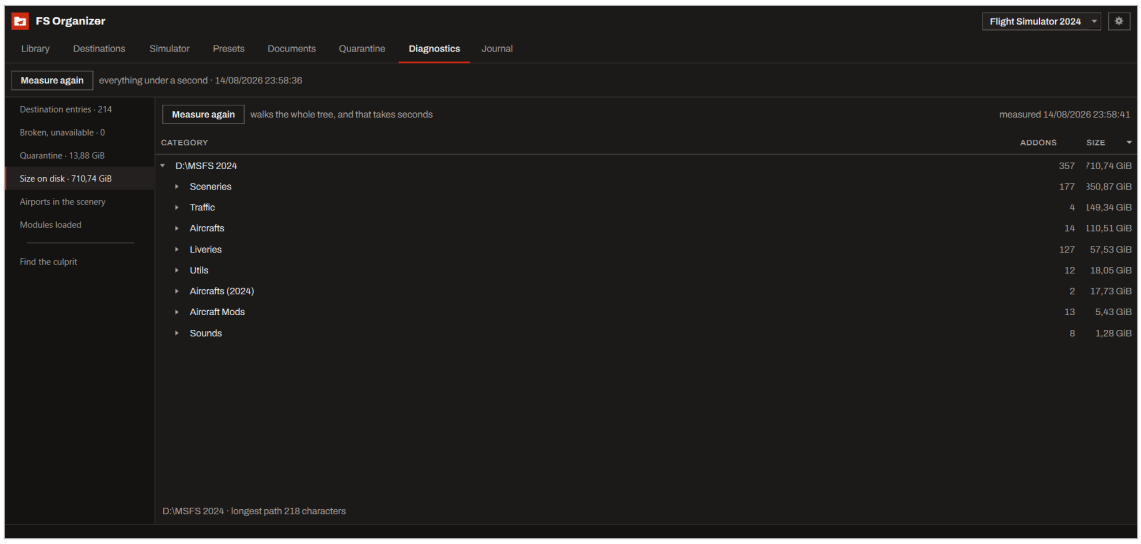


The documents tab: charts grouped by airport on the left, the reader in the middle and the outline of the document on the right.

Charts and manuals start with different gestures: on a chart the wheel zooms and dragging moves the page; on a manual both start off. The two toggles on the bar say what they do.

11 Diagnostics

The **Diagnostics** tab answers for what the scan found. Every number carries the time it was measured, and measuring again is always one button away.



The size on disk section, with the total per category and per addon, and the time of the measurement.

Destination entries
How many entries there are, counted by what each one is, and the longest path of each library.

Broken, unavailable
What is broken and what is parked on a volume that is not here, with the repair for the broken ones right there.

Quarantine
How much is held, without emptying anything from here.

Size on disk
How much each category and each addon takes. It walks the whole tree, and that takes seconds: the **Stop** button stops after the addon being measured now.

Airports in the scenery
Reads the scenery of every addon and separates three cases: carrying an airport code, carrying a record that did not decode, and carrying no airport record.

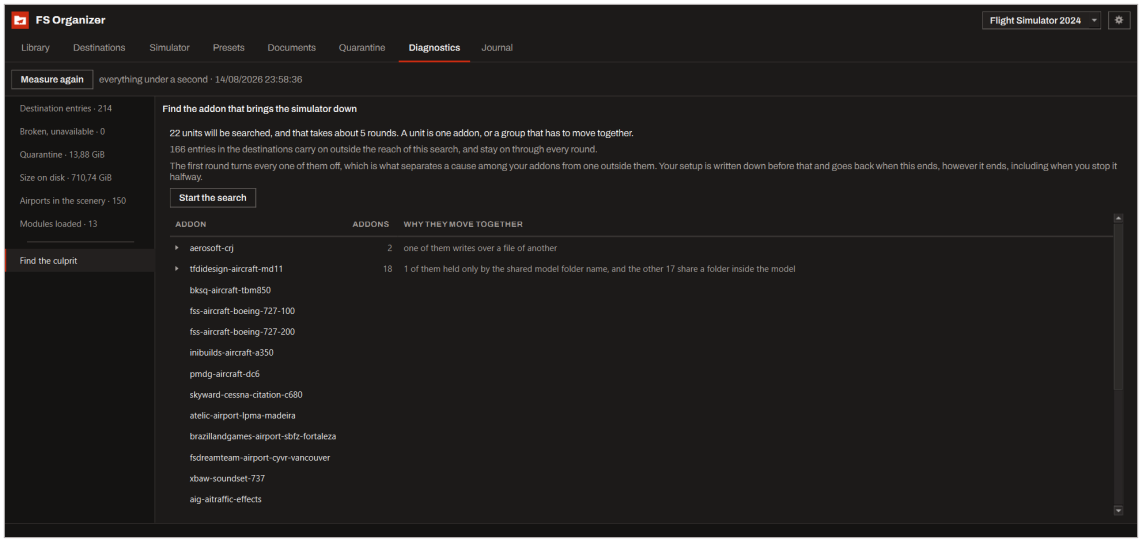
Modules the simulator loaded
Reads the report the simulator writes when a load takes long. It attributes no loading time to any package, so this screen shows none; what it does attribute is the module each package loaded and the memory that module holds.

11.1 Finding the culprit

When the simulator comes down and you do not know which addon brings it there, the **Find the culprit** panel runs a search by halves. It turns half of your addons off, you launch

the simulator and answer **It came down** or **It ran fine**, and the search rules out half of the suspects each round.

Three things to know before starting:



The search for the culprit, with the current round, what has been ruled out and the two answers the user gives.

- The unit of the search is not the addon, it is the **unit**: one addon, or a group that has to move together because it shares a model folder. The panel says why the members of a group move together, and offers to split the group when doing so leaves a state anyone can read.
- The first round turns **everything** of yours off. That round is what separates a cause among your addons from a cause outside them.
- Your setup is written down before the first round and goes back when this ends, however it ends, including when you stop it halfway. If the program is closed during the search, the next run offers to put it back.

The method assumes a single culprit. Two addons that only bring the simulator down when both are on would converge on an innocent one, and the panel says so.

12 Journal

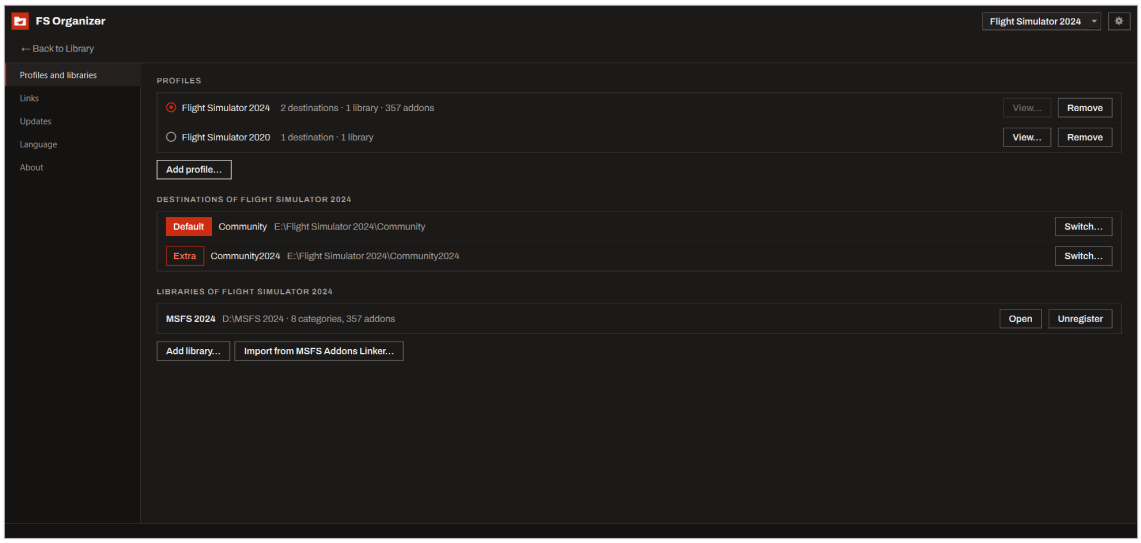
Every operation that writes to the disk becomes a line in the **Journal**. It is append-only, nothing on its screen writes to the disk, and **the program never deletes it**.

Each line carries when, which operation, which addon, which library, source, destination and result. The **Only what failed** filter narrows the list to what went wrong, and the search covers addon, path and operation.

Undo the last batch lives here and on the **Library** tab: it takes back the whole last batch, not one operation inside it.

13 Options

The gear in the header opens the options, in five tabs.



The profiles and libraries tab of the options.

Profiles and libraries
Adds and removes profiles, switches destinations, registers and unregisters libraries. Unregistering takes the library out of the configuration and deletes no file at all: the links that pointed at it keep working in the simulator, but start showing up as third party links, which the program does not touch. **Categories** says how many folders of that library already carry the category marker and how many would receive it, and marks them or takes every marker back.

Links
The link type and the check after copying.

Updates
Automatic, notify or manual. The automatic one downloads the new version on its own and applies it when you close the program.

Language
English and Brazilian Portuguese. The switch takes effect right away, with no restart.

About
The version, the licence and the credits.

14 Where things are kept

The whole configuration of the program lives under `%LOCALAPPDATA%\fs-organizer`, like this:

`settings.json` holds the profiles, the libraries, the destinations and your choices.

`journal\operations.jsonl` is the journal.

`presets` holds the presets, one file per preset, plus the return preset of each profile.

`bisection` holds the state of a search for the culprit that is under way. It stops existing when the search ends.

None of your addon files live there. Deleting that folder deletes the configuration and the presets, and touches no addon: the program rebuilds what it knows by reading the destinations and the libraries again.

One thing the program writes outside that folder: the hidden `.fsorg-category` it leaves in the folders of your library that group addons. It holds nothing, and its whole job is to keep a category counting as one after it empties. **Categories** in the options takes every one of them back.